SNIA SDC: StorageAI 2026 - AiSIO: Orchestrating Storage I/O Across CPUs and Accelerators

Metadata

Field	Value
Channel	SNIAVideo
Uploaded	2026-05-12
Duration	29m 34s
URL	https://www.youtube.com/watch?v=AJe22Nah5KA
Transcript source	manual (en)

Executive summary

A Samsung researcher presents AiSIO (Accelerator-integrated Storage I/O, also called ACIO), an open-source project that extends the Linux storage stack to allow NVMe storage I/O to target GPU accelerator memory directly, without requiring data to transit host DRAM. The talk proceeds in three parts: a diagnosis of three software bottlenecks limiting accelerator I/O today; a tour of a new architecture (replacing an earlier libvm-based prototype) built around HOMIE, the Extend Access Library, and the Ublock infrastructure; and benchmark results on a 16-NVMe-drive Dell server at Samsung Memory Research Center demonstrating that application- and driver-layer optimization can reduce the CPU cost of 62 million IOPS from 8 cores to 1.5 cores, and that peer-to-peer DMA to GPU memory incurs no additional NVMe processing cost compared to host-memory targets.

During the component overview, an audience member named Christoph Hellwig interrupted to object that the F_MAP_AP-based approach is a debugging tool carrying file-system-specific risk and corruption hazard under concurrent access, and later reiterated this concern in the Q&A. The main speaker acknowledged the objection and noted that PNF and flex files are also under consideration as management layers.

Topics covered

Introduction and Problem Statement (00:00:05–00:03:43)

The talk opens with a framing of AiSIO around three categories of software overhead that limit accelerator I/O performance. The stated design goal is interoperability over replacement: extend, not displace, the existing storage stack.

- Three challenges: (1) per-layer overhead across applications, libraries, language runtimes, user/kernel boundaries, and the OS storage stack; (2) unwanted data copies bouncing through host DRAM before reaching the accelerator; (3) the PCIe single physical function limit, which restricts a storage device to one associated driver.
- Three I/O modes targeted: CPU initiated (data to host memory), CPU initiated peer-to-peer (CPU drives, data lands in accelerator memory), and device initiated peer-to-peer (accelerator self-initiates, data to its own memory).
- Design philosophy stated as interoperability over replacement: extend the existing stack without trading off existing abstractions or performance.
- All components target upstream Linux kernel and open-source projects.
- Vision stated as making accelerators first-class citizens in the storage stack.

Related Work and Initial Prototype (00:03:43–00:08:22)

The speaker surveys existing solutions before describing the team's first prototype, which reproduced device-initiated I/O from BAM and libenvm and added file system support via SIOV-based multi-driver binding and a file extend cache.

- Existing solutions cited: Nvidia GDS, BAM (Big Accelerator Memory), Nvidia Scattera, AMD ROCk XIO, and libenvm.
- Nvidia GDS benchmark showed that a single thread at small I/O sizes could not saturate even a single NVMe device.
- The initial prototype used SIOV (multiple physical functions on one drive) to bind both a kernel stack driver and an ACIO prototype driver simultaneously.
- A file extend cache was built so the GPU could load LBAs directly without repeated filesystem lookups.
- Block I/O results: the ACIO prototype matched BAM; Nvidia GDS showed degraded performance especially at 96 threads.
- File throughput comparison across small and larger file workloads showed benefits for smaller files.
- Challenges with the prototype: codebase compliance with open-source stack, upstream feasibility, and file extent lookup reliability.

New AiSIO Architecture (00:08:22–00:11:46)

The new architecture eliminates libvm, custom kernel patches, and exclusive device ownership in favor of a daemon-based control plane and standard kernel interfaces.

- New architecture replaces libvm, custom kernel patches, and exclusive device ownership.
- HOMIE (Host Orchestrated Multipath IO Daemon): the control plane, brings up all I/O paths on the host.
- UDMA buff import: provides access to physical GPU memory addresses.
- Extend Access Library: caches file extent data inside HOMIE; replaced raw XFS on-disk format decoding with the kernel's F_MAP_AP interface.
- User-space processes access HOMIE through UIO PCI generic and VFIO PCI; a small ioctl taps into DMA buff and UDMA buff interfaces.
- For devices supporting SIOV or multiple physical functions, hardware-assisted multi-driver binding is used; for single-function

devices, the Linux Ublock infrastructure enables a software-based user-space control plane.
- EBPF traces are planned for the Extend Access Library to capture file change notifications alongside F_MAP_AP results.

Open Source Components (00:11:46–00:16:19)

The open-source component overview, presented as a component diagram with existing projects in blue and new or extended ones in orange, was interrupted by Christoph Hellwig's objection before the speaker continued with the remaining tools.

- XNME: user-space I/O library extended with AiSIO primitives; wherever XNME is integrated the new I/O paths become available.
- XME Perf: minimal-overhead benchmarking tool for CPU-initiated and device-initiated I/O patterns.
- UPC: MMIO doorbell module usable from both CPU and accelerator; abstracts PCIe doorbell ring for driver authors.
- DMA buff infrastructure tap: small module not yet upstream; made openly available for Linux kernel community feedback.
- Extend Access Library: evolved from raw XFS decode to F_MAP_AP backend to a planned F_MAP_AP plus EBPF combination for change notifications.
- Field Path: file-level benchmarking tool comparing POSIX, GDS CUFILE, and AiSIO interfaces.
- CHO: provisioning tool analogous to Ansible, enabling reproducible deployment of the full AiSIO stack.
- White paper and SDK are publicly available.

Benchmark Results (00:16:19–00:25:20)

Four experiments cover: a tuned CPU baseline, application-layer optimization, driver-layer optimization, and peer-to-peer GPU-targeted I/O (both CPU-initiated and device-initiated), followed by a bandwidth saturation analysis.

- Test system: 16 NVMe drives in a Dell server hosted at Samsung Memory Research Center; drives in unallocated mode (NVME LBA format) to eliminate media latency; per-drive unallocated IOPS: 3.8 million.
- Baseline with SPDK BDEV Perf: 62 million IOPS with 8 CPU cores saturating all 16 drives; scales linearly at 7.7 million IOPS per core; NUMA effects cause some variation.

- Application-layer comparison (same NVME driver, different front-ends): BDEV Perf yields 6.3M / 7.4M / 16M IOPS at 1 / 2 / 3 hardware threads; removing the BDEV layer (NVME Perf) shows a dramatic improvement; XME Perf via XNME further raises this to 32 million IOPS with 3 hardware threads.
- Driver-layer tweaking with UPCE tooling: approximately 40 million IOPS on 1 hardware thread; approximately 50 million IOPS on 1 CPU with 2 hardware threads; at this point the system is bottlenecked by device count, not software.
- Overall CPU efficiency gain: from 8 cores to 1.5 cores equivalent for 62 million IOPS.
- CPU initiated peer-to-peer (data to GPU): identical IOPS ceiling to host-memory baseline; NVMe drive DMA cost is the same whether the target is host memory or GPU memory; no additional CPU cost for peer-to-peer steering.
- Device initiated peer-to-peer: demonstrated as functional; reaches the same IOPS ceiling; GPU resource consumption for this mode not reported in slides.
- Peer-to-peer bandwidth: at 4K I/O size, 3 NVMe drives saturate the PCIe Gen 5 GPU link; adding a 4th drive yields minimal gain; PCIe protocol overhead is identified as the limiting factor.
- At 4K I/O size, a single CPU can saturate 8 GPUs in bandwidth terms.

Conclusion and Q&A (00:25:20–00:29:31)

The speaker summarizes the vision and takes audience questions; Christoph Hellwig delivers a fuller version of his earlier objection.

- All components open source, either already upstream or targeting upstream.
- New device-initiated experiments with additional results are described as available.
- Source code and white paper publicly available.
- Christoph Hellwig's Q&A objection: F_MAP_AP is a debugging tool with file-system-specific content; poses corruption risk under concurrent activity; he stated he raised this with the team and its management two years prior and recommended expanding PFS block layout with relocation mechanisms.
- Speaker's response: team also sees benefits in PNF and flex files as management and abstraction layer; F_MAP_AP is not used exclusively.
- Files in benchmark experiments: pre-allocated, fixed size.

- Data-loader workload: image dataset (including TikTok dataset) and 8 GB files accessed randomly from GPU, inspired by Nvidia DALI framework.

People, organizations, products, places

People

- Christoph Hellwig (audience member; objected to F_MAP_AP approach during talk and Q&A)

Organizations

- Samsung
- Nvidia
- AMD
- Dell
- SNIAVideo (channel)

Products

- AiSIO / ACIO (Accelerator-integrated Storage I/O)
- HOMIE (Host Orchestrated Multipath IO Daemon)
- Nvidia GDS (GPU Direct Storage)
- BAM (Big Accelerator Memory)
- Nvidia Scattera
- AMD ROCk XIO
- SPDK (BDEV Perf, NVME Perf)
- XNME
- XME Perf
- UPC
- UPCE
- Field Path
- CHO
- Extend Access Library
- Ublock
- libenvm
- Nvidia DALI

Places

- Samsung Memory Research Center (host site for Dell benchmark server)

Numbers and data points

Value	Context
3.8 million IOPS	Per-drive IOPS in unallocated (NVME LBA format) mode
62 million IOPS	Aggregate from 16 NVMe drives with 8 CPU cores (SPDK BDEV Perf baseline)
7.7 million IOPS	Per CPU core at baseline; scales linearly
6.3 million IOPS	BDEV Perf, 1 hardware thread
7.4 million IOPS	BDEV Perf, 2 hardware threads
16 million IOPS	BDEV Perf, 3 hardware threads on 2 CPU cores
32 million IOPS	XME Perf, 3 hardware threads (application-layer tweaking)
~40 million IOPS	Driver-layer tweaking (UPCE), single hardware thread
~50 million IOPS	Driver-layer tweaking (UPCE), single CPU / 2 hardware threads
1.5 cores	CPU cores equivalent after optimization to sustain 62 million IOPS
3 NVMe drives	Number needed to saturate PCIe Gen 5 GPU link at 4K I/O size
4K	I/O size at which PCIe Gen 5 GPU link is saturated by 3 drives
8 GPUs	Number a single CPU can saturate in bandwidth at 4K I/O
8 gigabyte	Size of large pre-allocated files in data-loader benchmark

Notable quotes

> basically we want to extend the existing software stack and we want to do so in ways where we're not trading off the utilities of having existing abstractions and we don't want to trade off performance either. — unknown

> that's the vision. We want to be able to have accelerators as uh uh first class citizens in the um in the storage stack. — unknown

> a single thread small IO we cannot saturate even a single NVME device using this. — unknown

> we hit the 62 million IOPS here with eight CPU cores. So that's 16 devices. Uh that gets saturated by using eight CPU cores. — unknown

> with three hardware threads you get 32 million IOPS. — unknown

> from these eight cores to re reach 62 million down to 1.5 — unknown

> the NVME drive doesn't really care that the DMA address is on your host memory or on your GPU. The cost of processing that IO is the same. — unknown

> is a proper way to do it and I told for two years to multiple members in your team how to do it and it's really upsetting that you can keep publishing kind of dangerous — Christoph Hellwig

> debugging tool. It has different content for different file systems. It's a massive risk for uh cost and corruption due to concurrent activity. That's why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients — Christoph Hellwig

> that was more a comment than a than a question and uh I think we'll take that into consideration. — unknown